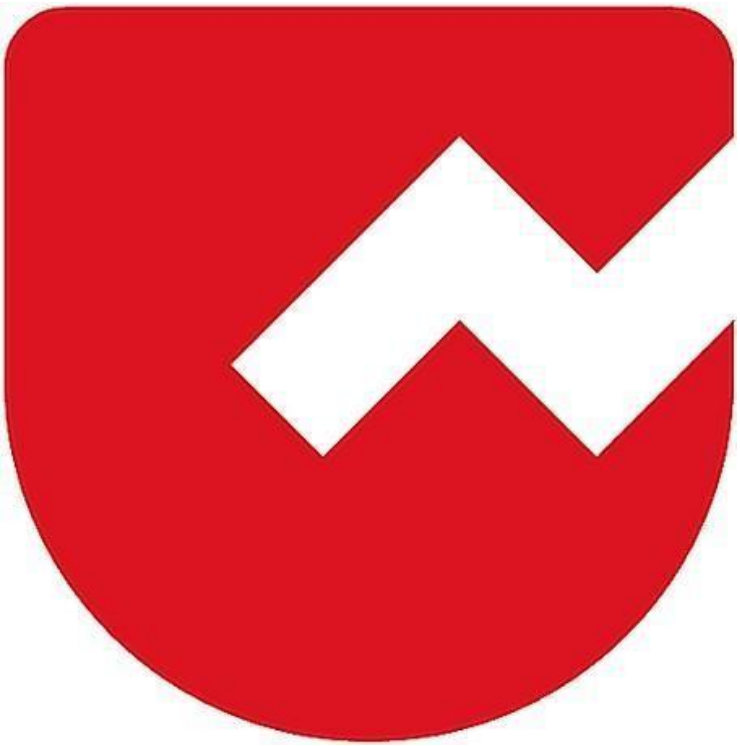
Zoho Schools for Graduate Studies



Notes

Bound wildcards in Generics

Session Date: December 3, 2025

1. Core Concepts

Generics vs. Arrays (Invariance vs. Covariance)

- **Arrays are Covariant:** Integer[] is a subtype of Number[]. You can assign an Integer array to a Number array reference.
- **Generics are Invariant:** List<Integer> is **NOT** a subtype of List<Number>.
  - *Reasoning:* If List<Integer> were a subtype of List<Number>, you could dangerously add a Double to a list of Integers via the Number reference, causing runtime errors.

Type Erasure

- **Definition:** Generics are a **compile-time** safety feature. The Java Virtual Machine (JVM) does not know about generic types at runtime.
- **Mechanism:** During compilation, the Java compiler validates type safety and then "erases" the specific type parameters (e.g., <Integer>), replacing them with their bound (e.g., Number) or Object.
- **Implication:** You cannot use generic types in runtime checks like instanceof T or create new instances like new T().

2. Bounded Type Parameters

To write flexible code that works with multiple specific types (like Integer, Double, Float), we use **Bounded Type Parameters**.

Syntax

public <T extends Number> void method(List<T> list)

- **<T extends Number>**: This declares a generic type T that is restricted to Number or its subclasses.
- **Benefit**: This allows you to call methods defined in the Number class (like doubleValue(), intValue()) on the generic objects inside the method.

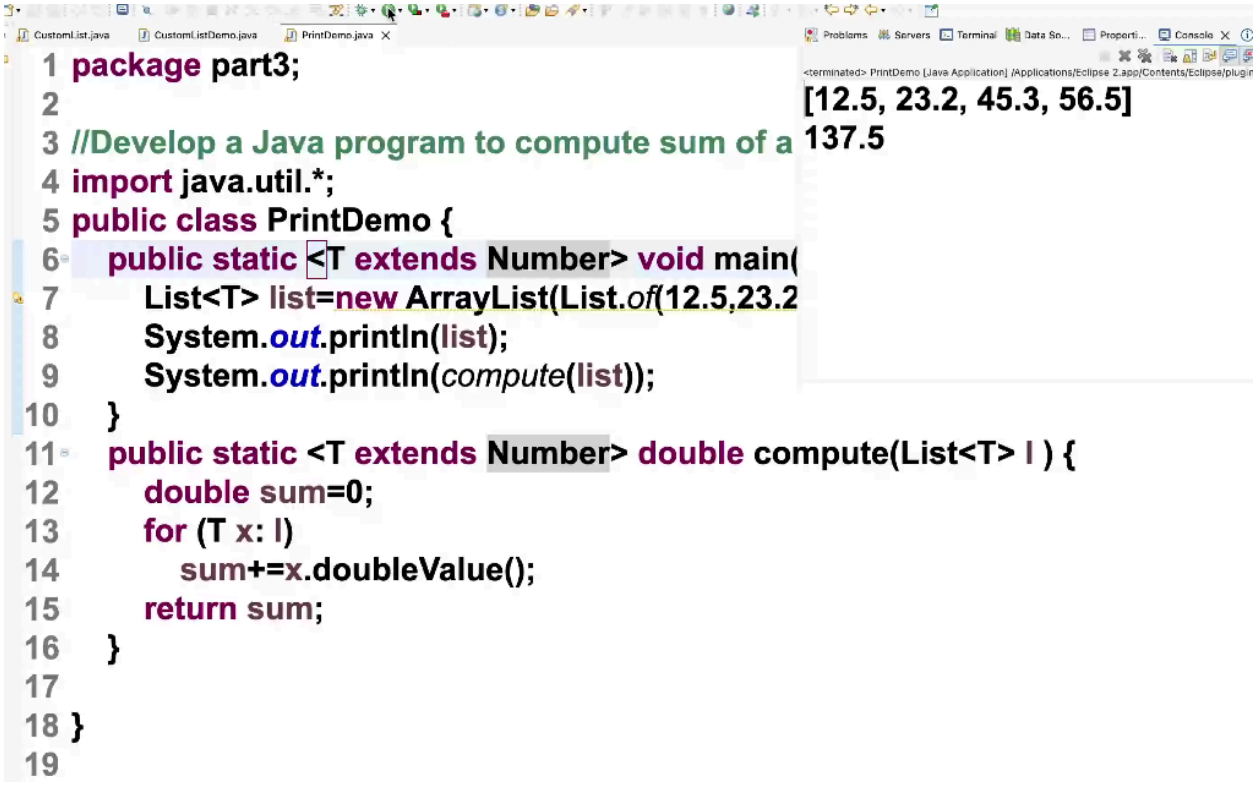
Unbounded vs. Bounded

- **Unbounded (<T> or <?>)**: Accepts any Java Object. Treated as Object inside the method.
- **Upper Bounded (<T extends Number> or <? extends Number>)**: Accepts Number and its subclasses. Treated as Number inside the method.

3. Practical Code Example: Sum of Numbers

Objective: Write a single method compute that calculates the sum of a list of numbers, regardless of whether they are Integers, Doubles, or Longs.

The Solution



Key Rules for Implementation

1. **Primitives are Forbidden:** You cannot use List<int> or List<double>. Generics only work with Reference Types (Objects). You must use wrapper classes like List<Integer> and List<Double>.
2. **Generic Declaration:** The <T extends Number> declaration must appear **before** the return type of the method.
3. The "Wildcard" Alternative: The method signature could also be written using wildcards: public static double compute(List<? extends Number> list)

4. Wildcards

Wildcards (?) allow for more flexible method parameters when the exact type is unknown or irrelevant.

Types of Wildcards

1. **Unbounded Wildcard (<?>)**:
   - Represents a list of *any* type.
   - Useful when the method only uses methods from the Object class or doesn't depend on the specific type parameter.
   - *Example:* void printList(List<?> list) can print a list of anything.

2. **Upper Bounded Wildcard (<? extends Number>)**:
   - Restricts the unknown type to be a specific type or a **subtype** of that type.
   - *Usage:* Useful for "reading" from a list where you know everything in the list is at least a Number.
   - *Example:* double sum(List<? extends Number> list) allows List<Integer>, List<Double>, etc.

```
package part3;

//Develop a Java program to compute sum of a given list of numbers
import java.util.*;
import java.io.*;
public class PrintDemo {
    public static void main(String[] args) {
        List<? extends Number> list=new ArrayList(List.of(12L,23L,45L,56L));
        System.out.println(list);
        System.out.println(compute(list));
    }
    public static  double compute(List<? extends Number> l ) {
        double sum=0;
        for (Number x: l)
            sum+=x.doubleValue();
        return sum;
    }
```

3. **Lower Bounded Wildcard (<? super Integer>)**:
    - Restricts the unknown type to be a specific type or a **super** type of that type.
    - *Usage:* Useful for "writing" to a list (e.g., adding Integers to a List<Number> or List<Object>).
    - *Example:* void addNumbers(List<? super Integer> list) allows List<Integer>, List<Number>, List<Object>.

Wildcards vs. Bounded Type Parameters

- **Bounded Parameter (<T extends Number>)**: Use when you need to refer to the type T multiple times (e.g., strictly ensuring the return type is T, or using T to declare variables inside the method).
- **Wildcard (<? extends Number>)**: Use when you only need to access the list and don't care about the specific type name inside the method body (often cleaner syntax).

5. Key Takeaways

1. **Code Reuse:** Generics prevent code replication (avoiding separate methods for sum(List<Integer>), sum(List<Double>), etc.).
2. **Type Safety:** Generics move type checking from runtime to compile-time, preventing ClassCastException.
3. **The "Number" Hierarchy:** Remember that Byte, Short, Integer, Long, Float, and Double all extend the abstract class Number.